Sprint 8: Asynchronous Apex (Future, Queueable, Batch & Scheduled Apex)

Introduction

As Salesforce applications scale up, certain operations start taking noticeably longer, and running them immediately can make the interface feel sluggish. Asynchronous Apex moves that kind of work into the background so users aren't stuck waiting on processing that isn't essential to their immediate task.

At its core, Asynchronous Apex exists to keep applications fast, scalable, and pleasant to use, even as the underlying business processes get more complex.

---

1. Synchronous vs Asynchronous Processing

Synchronous Processing

With synchronous processing, the user is stuck waiting until every operation in the transaction finishes.

Flow

User Request
↓
Execute Logic
↓
Query Data
↓
Validate Data
↓
Perform DML Operations
↓
Return Response
↓
User Continues

Example

When a student submits a job application, the system must immediately:

- Validate eligibility
- Check for duplicate applications
- Save the application
- Display a confirmation message

These operations must complete before the user receives a response.

Asynchronous Processing

With asynchronous processing, only the essential work happens right away — everything else is queued up and handled in the background afterward.

Flow

User Request
↓
Perform Essential Work
↓
Return Response to User
↓
Background Processing Continues

Example

After a student submits an application:

- Send acknowledgement email
- Update analytics
- Notify external systems
- Generate audit records

The student does not need to wait for these tasks.

---

2. Why Use Asynchronous Apex?

Asynchronous Apex is useful when:

- Processing takes a long time
- External systems need to be contacted
- Large datasets must be handled
- Tasks should run at scheduled times
- Users should not wait unnecessarily

Benefits

- Better user experience
- Improved performance

- Reduced transaction time
- Better handling of large datasets
- Scalability

---

3. Future Methods

Overview

Future Methods were Salesforce's original take on asynchronous processing and remain one of the simplest tools available.

The marked code runs separately, kicking off only once the triggering transaction has finished.

Syntax

```
@future
public static void processApplicationAsync(Id applicationId) {
    // Background processing
}
```

Example Use Case

After an application is saved:

- Send data to an external system
- Perform non-critical updates

Advantages

- Simple to implement
- Supports asynchronous callouts

Limitations

- Limited flexibility
- Cannot easily chain jobs
- Less powerful than Queueable Apex

---

4. Queueable Apex

Overview

Queueable Apex takes a more disciplined approach to background work compared to Future Methods.

A class implementing Queueable becomes a background job that Salesforce adds to a queue and runs when resources are available.

Syntax

```
public class OfferProcessingJob implements Queueable {


    public void execute(QueueableContext context) {


        // Background processing


    }
}
```

Enqueue Job

```
System.enqueueJob(
    new OfferProcessingJob()
);
```

Use Cases

- External integrations
- Notification processing
- Audit record creation
- Multi-step background workflows

Advantages

- More flexible than Future Methods
- Supports complex processing
- Supports job chaining
- Easier maintenance and testing

---

5. Future Methods vs Queueable Apex

Feature	Future Method	Queueable Apex
Background Processing	Yes	Yes
Pass Complex Data	No	Yes
Job Chaining	No	Yes
Structured Design	Limited	Strong
Recommended for New Projects	No	Yes

Recommendation

For most modern Salesforce projects, Queueable Apex is preferred over Future Methods.

---

6. Queueable Chaining

Overview

One Queueable job can start another Queueable job after completing its work.

Example

Offer Accepted
↓
External Synchronization Job
↓
Notification Processing Job

Benefits

- Separates responsibilities
- Creates cleaner architecture
- Improves maintainability

Considerations

Developers should think about:

- Failure handling
- Duplicate execution
- Job sequencing
- Monitoring

---

7. Batch Apex

Overview

Batch Apex comes into play when a job needs to work through a very large number of records.

Rather than trying to process everything in a single transaction, Salesforce splits the records into smaller, manageable batches.

Example

Processing:

- 80,000 applications
- 120,000 historical records
- Large analytics calculations

Architecture

Large Dataset
↓
Batch 1
↓
Batch 2
↓
Batch 3
↓
Finish

---

8. Batch Apex Structure

A Batch Apex class generally contains three methods:

Start Method

Determines which records should be processed.

```
public Database.QueryLocator start(
    Database.BatchableContext context
)
```

Execute Method

Processes a subset of records.

```
public void execute(
    Database.BatchableContext context,
    List<SObject> scope
)
```

Finish Method

Executes after all batches complete.

```
public void finish(
    Database.BatchableContext context
)
```

---

9. Sample Batch Apex Structure

```
public class PlacementAnalyticsBatch
implements Database.Batchable<SObject> {

    public Database.QueryLocator start(
        Database.BatchableContext context
    ) {
        return Database.getQueryLocator(
            'SELECT Id FROM Application__c'
        );
    }

    public void execute(
        Database.BatchableContext context,
        List<Application__c> scope
    ) {

        // Process records
```

```
        update scope;
    }

    public void finish(
        Database.BatchableContext context
    ) {

        // Completion activities

    }
}
```

---

10. Batch Size Considerations

The ideal batch size depends on:

- CPU usage
- Query complexity
- Heap size
- DML operations
- Business logic complexity
- Callouts

There is no universal batch size suitable for every use case.

---

11. Scheduled Apex

Overview

Scheduled Apex lets a piece of code kick off on its own at a time you define in advance.

Example

Every day at 6:00 AM:

- Find expired jobs
- Update job status

Syntax

```
public class ExpiredJobScheduler
implements Schedulable {


   public void execute(
      SchedulableContext context
   ) {


      // Scheduled processing


   }
}
```

---

12. When to Use Scheduled Apex

Use Scheduled Apex when:

- Daily processing is required
- Weekly reports must be generated
- Periodic cleanup jobs are needed
- Time itself triggers the business process

---

13. Combining Scheduled Apex and Batch Apex

For large recurring workloads, Scheduled Apex can start a Batch Apex job.

Architecture

Scheduled Apex
↓
Start Batch Apex

↓
Process Large Dataset

Example

Every Sunday at midnight:

- Start a Batch Job
- Process 150,000 historical records
- Generate analytics

---

14. Governor Limits in Asynchronous Apex

Asynchronous Apex still has Governor Limits.

Developers must continue following best practices:

Avoid

- SOQL inside loops
- DML inside loops
- Excessive CPU usage
- Large memory consumption

Follow

- Bulkification
- Efficient queries
- Efficient DML operations

Moving code to the background does not eliminate performance problems.

---

15. Idempotency

Definition

An operation is considered idempotent when running it more than once still leaves the business outcome unchanged.

Example

Suppose an external integration job executes twice.

Potential issues:

- Duplicate records
- Duplicate notifications
- Incorrect analytics

Developers should design systems that safely handle retries and duplicate executions.

---

16. Monitoring Asynchronous Jobs

Salesforce keeps a record of every asynchronous job through the AsyncApexJob object.

Administrators can monitor:

- Submitted jobs
- Running jobs
- Completed jobs
- Failed jobs
- Processing statistics

Monitoring is critical for production environments.

---

17. Error Handling

No enterprise system is immune to the occasional failure.

Possible issues include:

- Network failures
- External system outages
- Invalid data
- Governor Limit exceptions

Developers should design systems to:

- Log errors
- Notify administrators
- Support investigation
- Enable safe retries

---

18. Choosing the Correct Asynchronous Tool

Requirement	Recommended Solution
Immediate validation	Synchronous Apex
Simple background processing	Future Method
Structured background processing	Queueable Apex
Large-volume processing	Batch Apex
Time-based execution	Scheduled Apex
Large recurring processing	Scheduled + Batch Apex

Conclusion

Asynchronous Apex gives Salesforce applications a way to handle background processing without sacrificing responsiveness or the ability to scale.

The four major asynchronous technologies are:

- Future Methods
- Queueable Apex
- Batch Apex
- Scheduled Apex

Being a good Salesforce developer isn't just about knowing how to write the code — it's also about knowing when that code should actually run.

The most important architectural question is:

Should this work happen now, later, in batches, or at a scheduled time?